

Khwopa College of Engineering
OS Lab Sheet
II Year/ II Part
Faculty: Computer and Electronics
Labsheet #3: Process Management in Linux

Objectives

- Understand the concept of processes in Linux.
- Learn how to create, view, and manage processes.
- Practice using system calls like `fork()`, `exec()`, and `wait()`.
- Differentiate between foreground and background processes.
- Learn how to kill or suspend processes.

Pre-requisites

- Basic knowledge of Linux commands.
- Understanding of C programming and shell scripting.
- Concepts of parent and child processes.

Theory

What is a Process?

A process is a program in execution. It has a **process ID (PID)**, memory space, execution state, and associated system resources.

Process States:

- **Running:** Executing on the CPU.
- **Ready:** Waiting to be assigned to the CPU.
- **Blocked:** Waiting for an event (I/O, etc.).
- **Terminated:** Finished execution.

Important Linux Commands:

Command	Description
<code>ps</code>	Displays currently running processes
<code>top / htop</code>	Dynamic real-time process viewer
<code>kill <PID></code>	Terminates a process
<code>nice / renice</code>	Sets the priority of a process
<code>&</code>	Run a process in the background
<code>fg / bg</code>	Resume a process in foreground/background

Part A: Linux Commands Practice

Task 1: Process Inspection

```
``  
  
ps aux  
top  
``
```

- View all active processes.
- Identify PID, CPU%, and MEM%.

Task 2: Background Process

```
`` `  
  
gedit &  
jobs  
fg %1  
  
`` `
```

- Run `gedit` in the background.
- Use `jobs` to see background jobs.
- Use `fg` to bring a job to the foreground.

Task 3: Killing a Process

```
`` `  
  
ps aux | grep gedit  
kill -9 <PID>  
`` `
```

- Find the PID of `gedit` and terminate it.

Part B: C Program – Fork, Exec, Wait

Example Program 1: Creating a Child Process using `fork()`

```
#include <stdio.h>  
#include <unistd.h>  
  
int main() {  
    pid_t pid = fork();  
    if (pid == 0) {  
        printf("Child Process ID: %d\n", getpid());  
    } else if (pid > 0) {  
        printf("Parent Process ID: %d\n", getpid());  
    } else {  
        printf("Fork failed!\n");  
    }  
    return 0;  
}
```

Example Program 2: Using `exec()`

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Running ls with exec\n");
    execl("/bin/ls", "ls", "-l", NULL);
    printf("This line won't be printed if exec is successful\n");
    return 0;
}
```

Example Program 3: Using `wait()`

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        printf("Child Process\n");
        sleep(2);
    } else {
        wait(NULL);
        printf("Child Complete. Parent Resumes.\n");
    }
    return 0;
}
```

Task Assignment:

Task 1:

Write a C program to:

- Create a child process.
- In the child process, use `exec()` to replace it with the `date` command.
- Parent waits for the child process to complete.

Task 2:

Write a shell script that:

- Starts a long-running process in the background.
- Uses `ps` to get its PID.
- Sleeps for 10 seconds and then kills the process.

Task 3:

Experiment with `nice` and `renice` to start a process with low priority and change it later.

Deliverables

Students must submit:

- Output screenshots or logs.
- Source code files (`.c` or `.sh`).
- A short observation/report summarizing what they learned.

References

- Linux Man Pages: `man fork`, `man exec`, `man ps`, `man kill`
- [The Linux Programming Interface by Michael Kerrisk]
- [<https://man7.org/linux/man-pages/>]